Latest updates: https://dl.acm.org/doi/10.1145/882262.882335

PDF Download
882262.882335.pdf
17 March 2026
Total Citations: 93
Total Downloads: 2390

Published: 01 July 2003

Citation in BibTeX format

ARTICLE

Smoke simulation for large scale phenomena

NICK RASMUSSEN

DUC QUANG NGUYEN, Stanford University, Stanford, CA, United States

WILLI GEIGER

RONALD FEDKIW, Stanford University, Stanford, CA, United States

Open Access Support provided by:

Stanford University

ACM Transactions on Graphics (TOG), Volume 22, Issue 3 (July 2003)
https://doi.org/10.1145/882262.882335
EISSN: 1557-7368

Smoke Simulation For Large Scale Phenomena

Nick Rasmussen
Industrial Light + Magic
nick@ilm.com

Duc Quang Nguyen
Stanford University
Industrial Light + Magic
dqnguyen@stanford.edu

Willi Geiger
Industrial Light + Magic
wgeiger@ilm.com

Ronald Fedkiw
Stanford University
Industrial Light + Magic
fedkiw@cs.stanford.edu

Abstract

In this paper, we present an efficient method for simulating highly detailed large scale participating media such as the nuclear explosions shown in figure 1. We capture this phenomena by simulating the motion of particles in a fluid dynamics generated velocity field. A novel aspect of this paper is the creation of highly detailed three-dimensional turbulent velocity fields at interactive rates using a low to moderate amount of memory. The key idea is the combination of two-dimensional high resolution physically based flow fields with a moderate sized three-dimensional Kolmogorov velocity field tiled periodically in space.

CR Categories: I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling—Physically based modeling;

Keywords: smoke, incompressible Navier-Stokes equations, Kolmogorov spectrum, wind fields

1 Introduction

Although numerical simulations of natural phenomena such as smoke, fire and water are now routinely used in the special effects industry, some of the larger scale phenomena remain challenging. For example, it is difficult to simulate the nuclear destruction of an entire city with the level of detail necessary for a feature film. Moreover, reference video footage tends to be of low quality and resolution, so computer simulations of these phenomena are preferable.

Previous method such as [Foster and Metaxas 1997; Stam 1999; Fedkiw et al. 2001] can produce near real time results on small grids and interactive results on moderate sized grids. However, on very large grids, e.g. $2000 \times 2000 \times 2000$, these methods are impractical. In fact, a grid of this size requires about 120GB of memory just to store the density and velocity field as floats, which is well beyond the capability of a high end work station. And while large parallel computers may be available to some users, the algorithms tend to scale rather poorly anyway.

Instead of using voxel grids defined throughout space, a more efficient use of memory can be obtained via particle methods. We are not referring to the particles methods such as [Foster and Metaxas 1996; Foster and Fedkiw 2001; Enright et al. 2002] that still require a three-dimensional grid to store the velocity fields, but instead methods that are grid independent such as Smoothed Particle Hydrodynamics (SPH), see e.g. [Gingold and Monaghan 1977; Desbrun and Cani 1996; Hadap and Magnenat-Thalmann 2001]. Unfortunately, the standard SPH approach can be rather expensive when using a large number of particles, since one has to keep track of the nearest neighbors and solve fluid equations for velocity and pressure. As an alternative, we advocate the integration of noninteracting particles forward in time using a wind field that does not require a large three-dimensional grid for its representation. Traditionally (e.g. [Sims 1990; Wejchert and Haumann 1991; Rudolf and Raczkowski 2000]) wind fields were created using the superposition principle inherent to solutions of a simplified Laplace equation model, and then either particles or a grid based density was advected through these fields. We instead propose deriving a velocity field richer in structure from a number of two-dimensional fluid dynamics simulations, which can be carried out rather efficiently even for a high level of detail. Thus to obtain results on the scale of $2000 \times 2000 \times 2000$, we only need to simulate a few 2000×2000 grids saving about a factor of 2000 in both simulation time and memory. This reduces the amount of memory to around 60 megabytes without sacrificing detail. Of course, we still need to store the position of the particles but only those that participate in the scene, i.e. since the particles don't interact, we don't need the ones deep inside (or behind) a plume. This is similar to the optimization of storing the particles only near a water surface proposed in [Foster and Fedkiw 2001].

Figure 1: **CG generated nuclear explosion.**

We also use a periodic spatial tiling of a moderate sized three-dimensional Kolmogorov spectrum, see e.g. [Stam and Fiume 1993; Lamorlette and Foster 2002], and this requires another 24 megabytes of memory. The main role of the Kolmogorov spectrum is to break up any two-dimensional artifacts and transition the particle motion into a three-dimensional turbulent structure, *not* simply to add more detail to the simulation. Both the two-dimensional computational fluid dynamics (CFD) solution and the Kolmogorov

Permission to make digital/hard copy of part of all of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication, and its date appear, and notice is given that copying is by permission of ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.
© 2003 ACM 0730-0301/03/0700-0703 $5.00

velocity field can be precomputed rather quickly (just a few seconds per frame).

The particles are rendered using a view-dependent voxel-based method that is efficient enough to allow sequences of several hundred frames to be rendered overnight at resolutions of over 2000 pixels across while still including all the effects necessary for visual realism. Details in the variation of density and illumination as small as a single pixel are visible. The volume is motion blurred according to both its motion and the camera motion. Direct illumination of the volume by external light sources with correct shadowing is supported, as is incandescence. Diffuse scattering of light within a participating medium is simulated as well.

2 Previous Work

Early approaches focused on the smoke's density (mostly ignoring velocity) [Gardner 1985; Perlin 1985; Ebert and Parent 1990; Sakas 1990], and detail was added using time animated solid textures. [Stam and Fiume 1993] modeled random velocity fields using a Kolmogorov spectrum, and [Stam and Fiume 1995] proposed an advection-diffusion approach for densities composed of "warped blobs" to model gaseous distortions by wind fields. In [Sakas 1993], the spectral systhesis approach was used to generate a three dimensional random density field for gaseous phenomena.

[Kajiya and von Herzen 1984] were the first to simulate the equations of fluid dynamics directly, and some two-dimensional models were considered in [Yaeger and Upson 1986; Gamito 1995], but one of the more significant works in three-dimensional simulations was [Foster and Metaxas 1996; Foster and Metaxas 1997]. Since their explicit time integration scheme limits the time step increasing the computational cost, [Stam 1999] proposed an unconditionally stable method that uses a semi-Lagrangian advection scheme. [Fedkiw et al. 2001] introduced the notion of a vorticity confinement [Steinhoff and Underhill 1994] to add more small scale rolling motions to these simulations.

The compressible version of the Navier-Stokes equations can be used to model explosions, see e.g. [Neff and Fiume 1999; Yngve et al. 2000]. Since solving the compressible equations requires a very small time step, others have modeled fire (without shock waves) using the incompressible version of the equations. For example, [Nguyen et al. 2002] simulated fire using two phase incompressible flow, and [Lamorlette and Foster 2002] proposed a very practical procedural approach.

Another large scale problem (but one that we do not pursue in this paper) is the simulation of clouds. See for example, [Dobashi et al. 2000] who used lattice gas solvers based on cellular automata, and [Miyazaki et al. 2002] who used an approach similar to [Fedkiw et al. 2001] including vorticity confinement.

There are several approaches to the problem of high-quality volume rendering, see e.g. [Brodlie and Wood 2001]. Early work [Blinn 1982; Kajiya and von Herzen 1984] traced rays directly through a set of spherical particles, however, it is difficult to support the high albedo scattering that occurs in an optically dense medium with this method. Other work used a three-dimensional grid with density and/or lighting information stored in each voxel, see e.g. [Levoy 1988; Stam 1999; Fedkiw et al. 2001]. Isotropic scattering can be calculated by exchange of light between the voxels. The problem with this method is that a large number of voxels are required to represent fine details in the volume particularly if features at varying distances to the camera are visible in a perspective projection. A related approach used "splatting" [Westover 1990], where instead of solving the volume rendering integral in three dimensions, the voxels are projected onto the image plane and composited in two dimensions. This technique offers potential advantages in speed and memory requirements, but also poses several difficulties, particularly in achieving high quality results with an animated perspective view. Some recent work [Mueller et al. 1999] has ameliorated these problems, but the technique is still best suited to visualization of scientific data.

Recent work used photon mapping [Jensen and Christensen 1998; Fedkiw et al. 2001] to simulate light transport in a participating medium. This uses bidirectional ray tracing with a structure of spatially sorted particles (photons) to store indirect illumination. The method is general enough to handle both isotropic and non-isotropic scattering and volume caustics. However, as the resolution of the rendered image and the detail present in the density of the volume and the lighting increases, the method becomes excessively memory and processor intensive. There is also no explicit mechanism to store density, direct illumination or self-illumination, although the method could be extended to do so.

3 Simulation Method

3.1 Two-dimensional Incompressible Flow

We model the smoke's velocity, $\mathbf{u} = (u,v)$, with the two-dimensional incompressible Euler equations [Landau and Lifshitz 1998]

$$\nabla \cdot \mathbf{u} = 0 \quad (1)$$

$$\mathbf{u}_t = -(\mathbf{u} \cdot \nabla)\mathbf{u} - \nabla p + \mathbf{f} \quad (2)$$

where p is the pressure of the gas, and $\mathbf{f}$ accounts for the external forces. Note that we have arbitrarily set the constant density of the fluid to one. Equation 2 is solved by first computing an intermediate velocity $\mathbf{u}^*$ ignoring the pressure term, and then adding the pressure correction term using $\mathbf{u} = \mathbf{u}^* - \Delta t \nabla p$ where the pressure is found by solving $\nabla^2 p = \nabla \cdot \mathbf{u}^* / \Delta t$. We use a semi-Lagrangian stable fluids approach to find the intermediate velocity $\mathbf{u}^*$ and solve the linear system of equations for the pressure using a preconditioned conjugate gradient method. See [Stam 1999; Fedkiw et al. 2001] for the details.

The smoke's temperature and density are passively convected by the velocity field, $T_t = -(\mathbf{u} \cdot \nabla)T$ and $\rho_t = -(\mathbf{u} \cdot \nabla)\rho$, and thus both can be solved for using the semi-Lagrangian stable fluids method. Heavy smoke tends to fall downwards due to gravity while hot gases tend to rise due to buoyancy. Although we don't account for the temperature in the simulation (only using it for rendering), the external buoyancy force is directly proportional to the density, $\mathbf{f}_{buoy} = -\alpha \rho \mathbf{z}$ where $\mathbf{z} = (0,1)$ points in the upward vertical direction and α is positive constant with appropriate units.

Nonphysical numerical dissipation damps out interesting flow features, and we use vorticity confinement (see [Steinhoff and Underhill 1994; Fedkiw et al. 2001; Nguyen et al. 2002]) to generate the swirling effects. First the vorticity $\omega = \nabla \times \mathbf{u}$ is identified as the ("paddle-wheel") source of this small scale structure, and then normalized vorticity location vectors, $\mathbf{N} = \nabla|\omega| / |\nabla|\omega||$ that point from lower to higher concentrations of vorticity are constructed. The magnitude and direction of the vorticity confinement force is computed as $\mathbf{f}_{conf} = \varepsilon h (\mathbf{N} \times \omega)$ where $\varepsilon > 0$ and is used to control the amount of small scale detail added back into the flow, and the dependence on the grid size h guarantees that the physically correct solution is obtained as the mesh is refined.

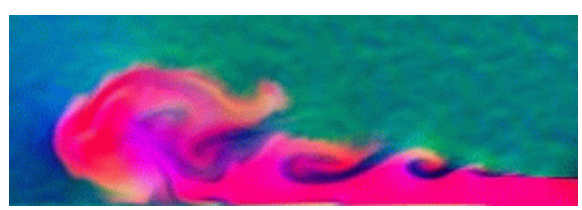

Figure 2: **Density contours from a two-dimensional simulation.**

3.2 Interpolation

After generating a few two-dimensional velocity fields (with density and temperature), we define a three-dimensional velocity field via interpolation. Traditional methods created wind fields using the superposition principle inherent to solutions of a simplified Laplace equation model, and then advected either particles or a grid based density through these fields. More recently there has been a surge of CFD algorithms, e.g. [Foster and Metaxas 1997; Stam 1999; Fedkiw et al. 2001; Nguyen et al. 2002] arising as competing methods. While wind fields are much faster than their CFD counterparts, the CFD algorithms produce highly detailed and realistic flow fields. Thus, we propose using the two-dimensional versions of these CFD algorithms as two-dimensional cross-sections of flow fields in three-dimensional space, and define interpolation methods to fill in the empty regions between these two-dimensional cross-sections. That is, we piece together velocity fields in three-dimensional physical space with the aid of interpolation, as opposed to piecing together analytic flow fields in solution space as is typical for wind field methods. Many of the benefits of both approaches are retained. For example, there is no need to solve the Navier-Stokes equations in three spatial dimensions. Only very efficient two-dimensional simulations are needed, and thus the bottle neck is the particle advection for both our method and the traditional wind field approach. Moreover, we obtain highly detailed physically based velocity fields from full, but two-dimensional, Navier-Stokes simulations removing the need for guessing how to piece together analytic flow fields to obtain the desired look.

For simplicity, suppose that we begin with two separate (but similar) two-dimensional computations and place them side by side in three-dimensional space as shown on the left hand side of figure 3. Then the velocity field (and temperature and density) at a point P can be defined using linear interpolation between the two closest points as shown in the figure. In general, these closest points will not lie exactly on two-dimensional grid points, so interpolation is again used to find an appropriate value at the closest point. Interpolation is carried out in time as well, i.e. the two-dimensional solutions are cached once per frame, and subframe values are defined at the closest points using interpolation in time. Although one cannot readily define flow fields throughout all of space using this method, the participating media phenomena we are concerned with usually lie in a finite region.

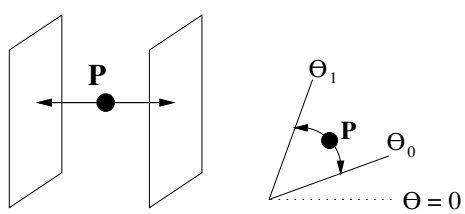

Figure 3: **Parallel interpolation (side view) and cylindrical interpolation (top view), respectively.**

There are many variants on this method, e.g. one could place the two-dimensional solutions in more creative ways, use different methods for interpolation, etc. The right hand side of figure 3 shows a top view of two planes tiled in a cylindrical fashion, and here the interpolation is carried out by finding the closest points along an arc of constant radius from the vertical axis. This variant is especially useful for the many phenomena that have approximate axial symmetry. For example, to generate a three-dimensional smoke plume, we calculate a number of similar (but slightly different) two-dimensional plumes, cut each one in half, and tile them in a circle at varying θ locations. Then the interpolation illustrated in figure 3 is carried out using the two slices that a point lies between. Note that one could use the axisymmetric Navier-Stokes equations here as well, but we have found this unnecessary especially since a number of lower dimensional calculations are required either way to provide a degree of variance from one tile to the next.

3.3 Kolmogorov Spectrum

Since we build a three-dimensional velocity field from two-dimensional solutions of Navier-Stokes equations, it is desirable to add a fully three-dimensional component to the velocity field. This is accomplished using a Kolmogorov spectrum which was described in detail in [Stam and Fiume 1993] and used in many application, e.g. to model fire in [Lamorlette and Foster 2002]. (Note that the Kolmogorov technique is similar to the Phillips spectrum techniques used by [Tessendorf 2002] to simulate water waves on oceans.) The main role of the Kolmogorov spectrum is *not* simply to add more detail to the simulation, but instead to break up any two-dimensional artifacts and transition the particle motion into a three-dimensional turbulent structure.

The main idea is to use random numbers to construct an energy spectrum in Fourier space that subsequently determines the structure of the velocity field. There are a wide variety of different models in the turbulence literature, but the most popular is probably the Kolmogorov energy spectrum

$$P_h(\mathbf{k}) = \begin{cases} 0 & if \quad k < k_{inertial} \\ 1.5\varepsilon^{\frac{2}{3}} k^{-\frac{5}{3}} & otherwise \end{cases}$$

where energy introduced at frequency $k_{inertial}$ is propagated to higher frequencies at a constant rate ε. After constructing an energy spectrum in Fourier space, one enforces the divergence free condition and uses an inverse FFT to obtain a velocity field full of small scale eddies. See [Stam and Fiume 1993] for more details.

Since the velocity field is periodic, a single grid can be used as a tiling of all of space. Moreover, one can use two grids of different sizes to increase the period of repetition to the least common multiple of their lengths alleviating visually troublesome spatial repetition (although this is a minor point for us since we blend the Kolmogorov velocity field with our non-periodic CFD/interpolation generated velocity field). We also fill the time domain by constructing a few Kolmogorov velocity fields, assigning each one to a different point in time, and interpolating between them at intermediate times. In practice, two spectrums are usually enough and we alternate between them every 24 frames. Finally, at any point in space and time, we define the total velocity field as a linear combination of the Kolmogorov field and the CFD/interpolation generated wind field. Again, we stress that we do not use this to construct a three-dimensional velocity field, but instead compute the velocity at a point in space and time on the fly using the Kolmogorov velocity field, the two dimensional CFD generated velocity fields, and appropriate interpolation rules.

3.4 Particle Advection

Once we have implicitly defined our flow field at every point of interest in space and time, we can passively advect particles through the flow using $\mathbf{x}_t = \mathbf{u}$ where $\mathbf{x}$ is the particle position. If desired, copies of two-dimensional flow fields and the three-dimensional Kolmogorov velocity field can be distributed to multiple processors where particles can be passively evolved with no intercommunication requirements. This allows one to generate an incredibly large numbers of particles, although we have found that even one processor can readily generate enough particles to move the bottleneck to the rendering stage.

There are many advantages to using particles to represent the flow field. For example, one can rapidly visualize the results of a calculation by simply drawing points at every particle location, density and temperature fields can be interpolated from the two-dimensional grids to the particle locations and stored there for subsequent rendering, and both an orientation and an angular velocity can be evolved with each particle to provide additional information

(such as an evolving coordinate system, see e.g. [Szeliski and Tonnesen 1992]) for the rendering stage.

4 Rendering

In order to alleviate the memory difficulties associated with mapping all the particles to a large three-dimensional voxel grid, we use a truncated pyramid shaped grid aligned with the view frustum, i.e. each voxel is a small truncated pyramid. This offers several advantages: as the grid is aligned with the pixels of the image, aliasing artifacts are greatly reduced; features near the camera are automatically resolved with the appropriate higher level of detail; and the resolution of the grid can be relatively low in the axis perpendicular to the view plane resulting in a smaller grid with a corresponding reduction in memory use and render time. In addition to the particle densities, the voxel grid also stores the total radiance (sum of direct illumination, incandescence and scattered light). The direct illumination and incandescence are calculated first, and then diffuse scattering of light within the volume is simulated.

4.1 Ray Marching

The volume is rendered by tracing rays from the camera through the voxel grid. As the voxel grid is aligned with the view frustum, ray traversal is extremely efficient (just incrementing an index). We accumulate color and opacity, where opacities of 0 and 1 correspond to zero and complete attenuation respectively. The opacity of a voxel centered at point $\mathbf{x}$ is calculated as $a = 1 - exp(-\tau D(\mathbf{x})dz)$ where $D(\mathbf{x})$ is the density of the voxel, dz is the depth of the voxel in the direction of the ray and τ is a constant that controls the conversion from density to opacity. We accumulate opacity along the ray by $A_{n+1} = A_n + a(1 - A_n)$. An advantage of ray tracing in this direction is that the trace can be terminated as soon as full opacity ($> .999$) is reached, and this happens relatively early for many of the phenomena that we are interested in. At each point we also accumulate the illuminated color of the volume weighted by the opacity calculated as above. This is $C_{n+1} = C_n + a(1 - A_n)I(\mathbf{x})$ where $I(\mathbf{x})$ is the stored illumination at the voxel. This gives us a correctly premultiplied image which can be directly composited over other elements in the scene.

4.2 Particle Sampling

The particle density is sampled onto the frustrum shaped voxel grid by treating each particle as one or more ellipsoids. Each ellipsoid is assigned a radius on each axis of its particles local coordinate system, and has a density at a point $\mathbf{x}$ in the local coordinate system given by $D(\mathbf{x}) = 1 - f(1-s, 1, |\mathbf{x}|/r)$ where $0 \le s \le 1$ is a softness factor, r is the particle radius, and f is defined as

$$f(a,b,t) = \begin{cases} 0 & if \quad t \le a \\ 1 & if \quad t \ge b \\ -2\left(\frac{t-a}{b-a}\right)^3 + 3\left(\frac{t-a}{b-a}\right)^2 & if \quad a < t < b \end{cases} .$$

A turbulence function is used to modulate the density function to add extra detail [Perlin 1985]. We also use motion blur to sample the ellipsoids onto the voxel grid according to the relative motion between the corresponding particle and the camera.

4.3 Lighting Model

Direct illumination of the volume is calculated at each voxel that has a non-zero density by tracing rays to each light source in the scene to calculate attenuation of the incoming light. The light from the source is attenuated as the ray is traced through the voxel grid using the method described in section 4.1. The volume can also be incandescent, and this is simulated by treating each particle as a small blackbody radiator that illuminates the surrounding voxels. The radius and shape of the illumination function is based on the density function of the particle and we allow the user explicit control over the mapping from blackbody temperature to color values. The self-illumination is then accumulated with the direct external illumination stored in the voxels.

Figure 4: **The left and middle explosion plumes were generated with our new 2D technique, while the plume on the right (shown for comparison) was generated with a fully 3D simulation.**

A final important effect is that of light scattering within the volume. The transfer of light in a participating medium is described by [Chandrasekhar 1960]. In general, the scattering function is anisotropic. However, in the case of a high albedo medium, it can be shown that that the scattering becomes effectively isotropic and can be modeled as a diffusion process [Stam and Fiume 1995; Jensen et al. 2002]. Having stored illumination values at each voxel, we then simulate isotropic light scattering as diffusion. We use a simple but efficient method that assumes that light is scattered from each voxel uniformly in all directions and attenuated exponentially with optical distance. Since a high albedo medium can scatter light over large distances, we accelerate this using a hierarchical method [Jensen and Buhler 2002].

5 Results

All of our simulations used a Pentium4 2.2GHz or comparable machine. Figure 4 (left and middle) shows a large explosion obtained by advecting a half million particles through a flow field constructed from two-dimensional 250×250 grid cell simulations and a $128 \times 128 \times 128$ Kolmogorov grid. The simulation times were a few seconds per frame each for the two-dimensional simulations, the Kolmogorov spectrum, and the particle advection. For comparison, figure 4 (right) shows the results obtained from a fully three-dimensional simulation (including a Kolmogorov spectrum) of an explosion with similar scale. Using a $140 \times 140 \times 220$ grid, the simulation used about 1.5 gigabytes of RAM and took about 1-2 minutes per frame. Figure 1 is characteristic of an even larger scale nuclear explosion, and we increased the resolution of the two dimensional simulations to 500×500 grid cells which still simulated in less than 10 seconds per frame. (We tried two-dimensional simulations as large as 2000×2000 grid cells and the computational cost was only about two minutes per frame.) While the Kolmogorov grid was the same size as in figure 4, we used as many as 6 million particles increasing the simulation times for particle advection to about two minutes per frame. Note that this example uses one radially interpolated set of velocity fields for the large plume and another for the ground elements. Unlike the smaller scale simulations in figure 4, we were unable to carry out a comparable simulation with a fully three-dimensional algorithm due to RAM limitations. Rendering times for all simulations ranged from 5-10 minutes per frame.

6 Conclusions and Future Work

In this paper, we modeled large scale phenomena using the combination of a few highly detailed two-dimensional flow fields and a

moderate sized three-dimensional Kolmogorov velocity field. This technique is a few orders of magnitude more efficient than fully three-dimensional fluid dynamics calculations, and thus can obtain a level of detail unattainable using those methods.

It is easy to sculpt and control flow fields in two spatial dimensions, since the results can often be obtained in real or interactive time. Moreover, one can create a library of two-dimensional simulations complete with the parameters used to generate them, and then an animator can more easily choose a starting point for constructing future simulations.

7 Acknowledgements

Research supported in part by an ONR YIP and PECASE award (ONR N00014-01-1-0620), a Packard Foundation Fellowship, a Sloan Research Fellowship, ONR N00014-03-1-0071, NSF ITR-0121288 and NSF DMS-0106694. We would like to thank Cliff Plumer, Steve Sullivan, Samir Hoon, Sebastian Marino and Industrial Light + Magic for for their support and enthusiasm.

References

BLINN, J. F. 1982. Light Reflection Functions for Simulation of Clouds and Dusty Surfaces. *Comput. Graph. 16*, 3, 21–29.

BRODLIE, K., AND WOOD, J. 2001. Recent Advances in Volume Visualization. *Comput. Graph. Forum 20*, 1, 125–148.

CHANDRASEKHAR, S. 1960. *Radiative Transfer*. Dover, New York.

DESBRUN, M., AND CANI, M.-P. 1996. Smoothed particles: A new paradigm for animating highly deformable bodies. In *Comput. Anim. and Sim. '96 (Proc. of EG Workshop on Anim. and Sim.)*, Springer-Verlag, R. Boulic and G. Hegron, Eds., 61–76. Published under the name Marie-Paule Gascuel.

DOBASHI, Y., KANEDA, K., OKITA, T., AND NISHITA, T. 2000. A Simple, Efficient Method for Realistic Animation of Clouds. In *SIGGRAPH 2000 Conf. Proc., Annual Conf. Series*, 19–28.

EBERT, D. S., AND PARENT, R. E. 1990. Rendering and Animation of Gaseous Phenomena by Combining Fast Volume and Scanline A-buffer Techniques. In *Proc. of SIGGRAPH 1990*, 357–366.

ENRIGHT, D., MARSCHNER, S., AND FEDKIW, R. 2002. Animation and Rendering of Complex Water Surfaces. In *Proc. of SIGGRAPH 2001*, 736–744.

FEDKIW, R., STAM, J., AND JENSEN, H. W. 2001. Visual Simulation of Smoke. In *Proc. of SIGGRAPH 2001*, 15–22.

FOSTER, N., AND FEDKIW, R. 2001. Practical Animation of Liquids. In *Proc. of SIGGRAPH 2001*, 23–30.

FOSTER, N., AND METAXAS, D. 1996. Realistic Animation of Liquids. *Graph. Models and Image Processing 58*, 471–483.

FOSTER, N., AND METAXAS, D. 1997. Modeling the Motion of a Hot, Turbulent Gas. In *Proc. of SIGGRAPH 1997*, 181–188.

GAMITO, M. N. 1995. Two dimensional Simulation of Gaseous Phenomena Using Vortex Particles. In *Proc. of the 6th Eurographics Workshop on Comput. Anim. and Sim.*, Springer-Verlag, 3–15.

GARDNER, G. Y. 1985. Visual Simulation of Clouds. In *Proc. of SIGGRAPH 1985*, 297–384.

GINGOLD, R. A., AND MONAGHAN, J. J. 1977. Smoothed Particle Hydrodynamics-Theory and application to nonspherical stars. *Mon. Not. R. Astron. Soc. 181*, 375.

HADAP, S., AND MAGNENAT-THALMANN, N. 2001. Modeling Dynamic Hair as a Continuum. *Comput. Graph. Forum 20*, 3.

JENSEN, H. W., AND BUHLER, J. 2002. A Rapid Hierarchical Rendering Technique for Translucent Materials. In *Proc. of SIGGRAPH 2002*, 576–581.

JENSEN, H. W., AND CHRISTENSEN, P. H. 1998. Efficient Simulation of Light Transport in Scenes with Participating Media using Photon Maps. In *Proc. of SIGGRAPH 2002*, 311–320.

JENSEN, H. W., MARSCHNER, S., LEVOY, M., AND HANRAHAN, P. 2002. A Practical Model for Subsurface Light Transport. In *Proc. of SIGGRAPH 2002*, 511–518.

KAJIYA, J. T., AND VON HERZEN, B. P. 1984. Ray Tracing Volume Densities. In *Proc. of SIGGRAPH 1984*, 165–174.

LAMORLETTE, A., AND FOSTER, N. 2002. Structural Modeling of Flames for a Production Environment. In *Proc. of SIGGRAPH 2002*, 729–735.

LANDAU, L. D., AND LIFSHITZ, E. M. 1998. *Fluid Mechanics, 2nd edition*. Butterworth-Heinemann, Oxford.

LEVOY, M. 1988. Display of Surfaces from Volume Data. *IEEE Comput. Graph. and Appl. 8*, 3, 29–37.

MIYAZAKI, R., DOBASHI, Y., AND NISHITA, T. 2002. Simulation of Cumuliform Clounds Based on Computational Fluid Dynamics. *Proc. Eurographics 2002 Short Presentation*, 405–410.

MUELLER, K., MOLLER, T., AND CRAWFIS, R. 1999. Splatting without Blur. In *Proc. IEEE Vis. 1999*, 363–370.

NEFF, M., AND FIUME, E. 1999. A Visual Model for Blast Waves and Fracture. In *Proc. of Graph. Interface 1999*, 193–202.

NGUYEN, D., FEDKIW, R., AND JENSEN, H. W. 2002. Physically Based Modeling and Animation of Fire. In *Proc. of SIGGRAPH 2002*, 721–728.

PERLIN, K. 1985. An Image Synthesizer. In *Proc. of SIGGRAPH 1985*, 287–296.

RUDOLF, M. J., AND RACZKOWSKI, J. 2000. Modeling the Motion of Dense Smoke in the Wind Field. *Comput. Graph. Forum 19*, 3.

SAKAS, G. 1990. Fast Rendering of Arbitrary Distributed Volume Densities. In *Proc. of Eurographics 1990*, 519–530.

SAKAS, G. 1993. Modeling and Animating Turbulent Gaseous Phenomena Using Spectral Synthesis. *The Vis. Comput. 9*, 200–212.

SIMS, K. 1990. Particle Animation and Rendering Using Data Parallel Computation. *Comput. Graph. 24*, 4, 405–413.

STAM, J., AND FIUME, E. 1993. Turbulent Wind Fields for Gaseous Phenomena. In *Proc. of SIGGRAPH 1993*, 369–376.

STAM, J., AND FIUME, E. 1995. Depicting Fire and Other Gaseous Phenomena Using Diffusion Process. In *Proc. of SIGGRAPH 1995*, 129–136.

STAM, J. 1999. Stable Fluids. In *SIGGRAPH 99 Conf. Proc., Annual Conf. Series*, 121–128.

STEINHOFF, J., AND UNDERHILL, D. 1994. Modification of the Euler Equations for "Vorticity Confinement": Application to the Computation of Interacting Vortex Rings. *Phys. of Fluids 6*, 8, 2738–2744.

SZELISKI, R., AND TONNESEN, D. 1992. Surface modeling with oriented particle systems. *Comp. Graph. (SIGGRAPH Proc.)*, 185–194.

TESSENDORF, J. 2002. Simulating Ocean Water. In *Simulating Nature: Realistic and Interactive Techniques*, SIGGRAPH 2002, Course Notes 9.

WEJCHERT, J., AND HAUMANN, D. 1991. Animation Aerodynamics. *Comput. Graph. 25*, 4, 19–22.

WESTOVER, L. 1990. Footprint Evaluation for Volume Rendering. In *Proc. of SIGGRAPH 1990*, 367–376.

YAEGER, L., AND UPSON, C. 1986. Combining Physical and Visual Simulation - Creation of the Planet Jupiter for the Film 2010. In *Proc. of SIGGRAPH 1986*, 85–93.

YNGVE, G. D., O'BRIEN, J. F., AND HODGINS, J. K. 2000. Animating Explosions. In *Proc. of SIGGRAPH 2000*, 29–36.